08 — tb_risc8_5stage.v Explained: The Testbench

Oct 6, 2026 · @shiv

1. Overview

`tb_risc8_5stage.v` tests the processor automatically. It makes a clock and a reset, runs the program built into imem.v, prints the pipeline every cycle, then checks every register, one memory byte and the flags, and prints PASS or FAIL. It is **simulation-only** code: it cannot become hardware.

Item	Value
File	sim_sources/tb_risc8_5stage.v (136 lines)
Module name	tb_risc8_5stage (has no ports: it is the top of the simulation)
Type	Testbench, simulation only
Added in Vivado as	Simulation Source, set as top for simulation
Calls	risc8_5stage_core, task check_reg, and built-in system tasks (see 1.2)
Called by	Nobody: the simulator starts here

1.1 Where it sits

```
tb_risc8_5stage.v      <== THIS FILE (top of the simulation)
└── risc8_5stage_core.v  (instance dut)
    ├── imem.v, control_unit.v, regfile.v,
    └── hazard_unit.v, alu.v, dmem.v
```

1.2 Everything this file calls, and where it belongs

Name	What it is	Defined in	Used at line(s)	Why
risc8_5stage_core	Module	risc8_5stage_core.v	46–53 (instance dut)	The design being tested
check_reg	Task	This file, lines 95–101	113–116	Compare one register with its expected value
$display	Built-in system task	Verilog language (simulator)	81, 97, 100, 112, 119–126, 128–129, 134	Print text to the console
$time	Built-in system function	Verilog language	82	Current simulation time
$finish	Built-in system task	Verilog language	130, 134	End the simulation

1.3 Internal signals it reads from other files (hierarchical references)

A testbench may look directly inside the design using dotted names such as `dut.u_rf.r[1]`. This works only in simulation.

Name used	Signal	Belongs to file	Lines used
dut.stall	Load-use stall	risc8_5stage_core.v (from hazard_unit.v)	74, 87
dut.take_branch	Branch taken	risc8_5stage_core.v	75, 88
dut.fwd_a_sel, dut.fwd_b_sel	Forwarding selects	risc8_5stage_core.v (from hazard_unit.v)	77, 89

Name used	Signal	Belongs to file	Lines used
dut.idex_valid	Real instruction in EX	risc8_5stage_core.v	77, 84, 89
dut.ifid_valid, dut.ifid_instr	IF/ID register	risc8_5stage_core.v	83
dut.exmem_valid	Real instruction in MEM	risc8_5stage_core.v	84
dut.memwb_reg_we, dut.memwb_rd, dut.memwb_data	MEM/WB register	risc8_5stage_core.v	85
dut.u_rf.r[...]	Register array R0–R7	regfile.v	96–97
dut.u_dmem.mem[8'h20]	Data memory byte 0x20	dmem.v	118–119

1.4 Verilog constructs used

Construct	Meaning in plain words
`reg clk = 0`	A variable with a starting value (simulation)
`initial forever #5 clk = ~clk`	Flip the clock every 5 ns, forever: a 100 MHz clock
`#22`	Wait 22 ns
`wait (halted)`	Pause until halted becomes 1
`@(posedge clk)`	Wait for the next rising clock edge
`task ... endtask`	A reusable piece of code that can contain $display and timing
`!==`	"Not identical": also catches unknown (x) values
`integer`	32-bit counter variable

2. Code walkthrough, part by part

2.1 File header (lines 1–30)

The header lists the six jobs of the testbench, the module it calls, the internal signals it reads (and which file each belongs to), and the four steps to run it in Vivado.

2.2 Signals and the call to the processor (lines 32–53)

```
`timescale 1ns/1ps

module tb_risc8_5stage;

    reg clk = 0;
    reg rst = 1;
    wire       halted;
    wire [7:0] pc;
    wire [1:0] flags;        // {Z, C} from the core
    wire       instr_done;   // 1 when an instruction finishes WB

    // CALL: module risc8_5stage_core from risc8_5stage_core.v
    risc8_5stage_core dut (
        .clk    (clk),
        .rst    (rst),
        .halted     (halted),
        .pc_out     (pc),
        .flags_out  (flags),
        .instr_done (instr_done)
    );
```

- The module has **no ports**: a testbench is the outermost block of the simulation.
- `clk` and `rst` are `reg` because the testbench drives them. `rst` starts at 1, so the processor begins in reset.
- **Module call:** `risc8_5stage_core dut (...)` creates the processor from **risc8_5stage_core.v**. "dut" means *device under test*. Its outputs are connected to wires so they can be watched.

2.3 Clock generator (lines 55–56)

```
    // 100 MHz clock (10 ns period): flip clk every 5 ns, forever
    initial forever #5 clk = ~clk;
```

Every 5 ns the clock is inverted. Two flips make one period of 10 ns, which is 100 MHz.

2.4 Counters (lines 58–66)

```
    integer cycles  = 0;   // cycles from reset release until halted
    integer retired = 0;   // valid instructions that reached WB
    integer stalls  = 0;   // load-use stall cycles
    integer flushes = 0;   // taken branches (each flushes 2 instructions)
    integer fwds    = 0;   // EX operands supplied by forwarding
    integer errors  = 0;
```

These count events for the performance summary at the end. `errors` counts failed checks.

2.5 The per-cycle monitor (lines 68–90)

```
    always @(posedge clk) if (!rst && !halted) begin
        cycles <= cycles + 1;
        if (instr_done)      retired <= retired + 1;
        if (dut.stall)       stalls  <= stalls  + 1;
        if (dut.take_branch) flushes <= flushes + 1;
        // hazard_unit.v only forwards into operands that are really used
        if (dut.idex_valid && (dut.fwd_a_sel != 2'b00 || dut.fwd_b_sel !=
2'b00))
            fwds <= fwds + 1;

        // IF/ID instruction shown in hex; '--' means bubble
        $display("t=%6t PC=%02h | ID:%s%04h | EX:v%b | MEM:v%b | WB:%s
R%0d<=%02h | Z=%b C=%b %s%s%s",
                 $time, pc,
                 dut.ifid_valid ? " " : "-", dut.ifid_instr,
                 dut.idex_valid, dut.exmem_valid,
                 dut.memwb_reg_we ? "w" : "-", dut.memwb_rd, dut.memwb_data,
                 flags[1], flags[0],
                 dut.stall       ? " STALL"           : "",
```

```
                dut.take_branch ? " BRANCH->flush 2" : "",
                (dut.idex_valid && (dut.fwd_a_sel != 2'b00 || dut.fwd_b_sel
!= 2'b00)) ? " fwd" : "");
    end
```

On every rising edge while the processor runs:

- **Counting:** it adds 1 to each counter whose event happened. `dut.stall`, `dut.take_branch` and `dut.fwd_a_sel` are **hierarchical references**: they reach inside the core to read internal signals (from risc8_5stage_core.v and hazard_unit.v).
- **Printing:** `$display` (built into Verilog) prints one line. `%02h` prints a 2-digit hex number, `%b` binary, `%s` text and `%6t` the time from `$time`.

2.6 Task check_reg (lines 92–101)

```
    // TASK: check one register (array inside regfile.v)
    task check_reg(input [2:0] r, input [7:0] exp);
        if (dut.u_rf.r[r] !== exp) begin
            $display("FAIL  R%0d = %02h, expected %02h", r, dut.u_rf.r[r],
exp);
            errors = errors + 1;
        end else
            $display("PASS  R%0d = %02h", r, exp);
    endtask
```

A **task** is a reusable block of code, like a function that may also print or wait. `check_reg` is **defined in this file** and called on lines 113–116. It reads register `r` directly from the array `r` inside **regfile.v** (`dut.u_rf.r[r]`), compares it with the expected value `exp`, and prints PASS or FAIL. `!==` also catches unknown (x) values.

2.7 Main test sequence (lines 103–131)

```
    initial begin
        #22 rst = 0;            // release reset after ~2 cycles
        wait (halted);          // run until HLT leaves the pipeline
        @(posedge clk); @(posedge clk);

        // Expected values come from the program in imem.v
        $display("\n------------- register / memory check -------------");
        check_reg(1, 8'h00);  check_reg(2, 8'h0F);
        check_reg(3, 8'h01);  check_reg(4, 8'h20);
```

```
        check_reg(5, 8'h00);  check_reg(6, 8'h66);
        check_reg(7, 8'h00);

        if (dut.u_dmem.mem[8'h20] !== 8'h0F) begin
            $display("FAIL  MEM[20] = %02h", dut.u_dmem.mem[8'h20]); errors =
errors + 1;
        end else $display("PASS  MEM[20] = 0F");

        if (flags !== 2'b11) begin
            $display("FAIL  flags Z=%b C=%b", flags[1], flags[0]); errors =
errors + 1;
        end else $display("PASS  flags Z=1 C=1");

        $display("\ncycles=%0d  instructions=%0d  CPI=%0.2f  load-use
stalls=%0d  taken branches=%0d  forwards=%0d",
                 cycles, retired, cycles * 1.0 / retired, stalls, flushes,
fwds);
        if (errors == 0) $display("*** ALL TESTS PASSED ***");
        else             $display("*** %0d TEST(S) FAILED ***", errors);
        $finish;
    end
```

1. After 22 ns, reset is released and the processor starts.
2. `wait (halted)` pauses until the program reaches HLT; then it waits two more clock edges.
3. It calls **check_reg** for R1–R7 with the values the program should produce.
4. It checks memory byte 0x20 inside **dmem.v** (`dut.u_dmem.mem[8'h20]`) and the flags.
5. It prints the performance summary, including CPI = cycles ÷ instructions, and the final verdict.
6. `$finish` (built into Verilog) ends the simulation.

2.8 Watchdog (lines 133–136)

```
    // Watchdog: stop if the CPU never halts
    initial begin #20000 $display("TIMEOUT"); $finish; end

endmodule
```

A second `initial` block runs at the same time. If the program has not finished after 20 µs (for example, an endless loop), it prints TIMEOUT and stops the simulation.

3. Expected output, changing the test, mistakes and viva

3.1 Expected output in the Vivado Tcl Console

One line per clock cycle, for example:

```
t= 25000 PC=00 | ID:-0000 | EX:v0 | MEM:v0 | WB:- R0<=00 | Z=0 C=0
t= 35000 PC=01 | ID: 823c | EX:v0 | MEM:v0 | WB:- R0<=00 | Z=0 C=0
t= 45000 PC=02 | ID: 840f | EX:v1 | MEM:v0 | WB:- R0<=00 | Z=0 C=0
```

Part of the line	Meaning
t=	Simulation time in picoseconds
PC=	Address being fetched
ID:	Instruction in IF/ID; a - in front means bubble
EX:v1 / MEM:v1	A real instruction is in that stage (v0 = bubble)
WB:w R1<=3c	R1 is being written with 0x3C (- = no write)
STALL, BRANCH->flush 2, fwd	A stall, a taken branch, or a forwarded operand in that cycle

The end of the output:

```
------------- register / memory check -------------
PASS  R1 = 00
PASS  R2 = 0f
PASS  R3 = 01
PASS  R4 = 20
PASS  R5 = 00
PASS  R6 = 66
PASS  R7 = 00
PASS  MEM[20] = 0F
PASS  flags Z=1 C=1

cycles=51  instructions=34  CPI=1.50  load-use stalls=1  taken branches=6
forwards=12
*** ALL TESTS PASSED ***
```

3.2 How to test a different program

1. Change the `rom[...]` lines in imem.v (see Document 03).
2. Work out by hand what R1–R7, the memory and the flags should be at the end.
3. Change the expected values in the `check_reg(...)` calls (lines 113–116), the memory check (line 118) and the flag check (line 122).

3.3 Common mistakes

- Setting the design file (risc8_5stage_core) as the simulation top instead of tb_risc8_5stage: nothing drives the clock.
- Trying to synthesize the testbench: `#` delays, `$display` and `initial forever` cannot become hardware.
- Renaming an instance (for example u_rf) in the core without updating the hierarchical names here.

3.4 Viva questions

Q1. What is a testbench? Simulation-only Verilog that drives the design's inputs (clock, reset) and checks its outputs. It is never synthesized.

Q2. Why does the testbench have no ports? It is the top of the simulation; nothing outside it connects to it.

Q3. What is a task, and where is check_reg defined? A task is reusable code that can print and wait. check_reg is defined in this same file (lines 95–101) and called on lines 113–116.

Q4. What are $display, $time and $finish? Built-in Verilog system tasks and functions provided by the simulator: print text, read the current time, and end the simulation.

Q5. How does the testbench see inside the processor? Through hierarchical references such as dut.u_rf.r[1], which name a signal by its path of instance names.

Q6. Why is there a watchdog? If the program never reaches HLT, the simulation would run forever. The watchdog stops it after 20 µs and prints TIMEOUT.